

Module 11 Deploying Data Integration Job Sequence

IBM InfoSphere Information Server for Data Integration



Module Objectives

After completing this topic, you should be able to:

- Use the DataStage Job Sequencer to build a job that controls a sequence of jobs
- Use Sequencer links and stages to control the sequence of a set of jobs
- Use Sequencer triggers and stages to control the conditions under which jobs run
- Pass information in job parameters from the master controlling job to the controlled jobs
- Enable restart
- Handle errors and exceptions

What is a Job Sequence?

- A master controlling job that controls the execution of a set of subordinate jobs
- Passes values to the subordinate job parameters
- Controls the order of execution (links)
- Specifies conditions under which the subordinate jobs get executed (triggers)
- Specifies complex flow of control
 - Loops
 - Wait for file
- Performs system activities
 - Email
 - Execute system commands and executables
- Can include Restart checkpoints

Basics for Creating a New Job Sequence

- Open a new job sequence
 - Specify whether its restartable
- Add stages
 - Stages to execute jobs
 - Stages to execute system commands and executables
 - Special purpose stages
- Add links
 - Specify the order in which stages are to be executed
- Specify triggers
 - Triggers specify the condition under which control passes across a link
- Specify error handling
- Enable / Disable restart checkpoints

Job Sequencer Stages

-  EndLoop Activity
-  Exception Handler
-  Execute Command
-  Job Activity
-  Nested Condition
-  Notification Activity
-  Routine Activity
-  Sequencer
-  StartLoop Activity
-  Terminator Activity
-  UserVariables Activity
-  Wait For File Activity

- Run stages
 - Job Activity: Run a job
 - Execute Command: Run a system command
 - Notification Activity: Send an email
- Flow control stages
 - Sequencer: Go if All / Any input links are TRUE
 - Wait for File: Go when file appears / disappears
 - StartLoop / EndLoop
 - Nested Condition: Go if condition satisfied
- Error handling
 - Exception Handler
 - Terminator
- Variables
 - User Variables

Example

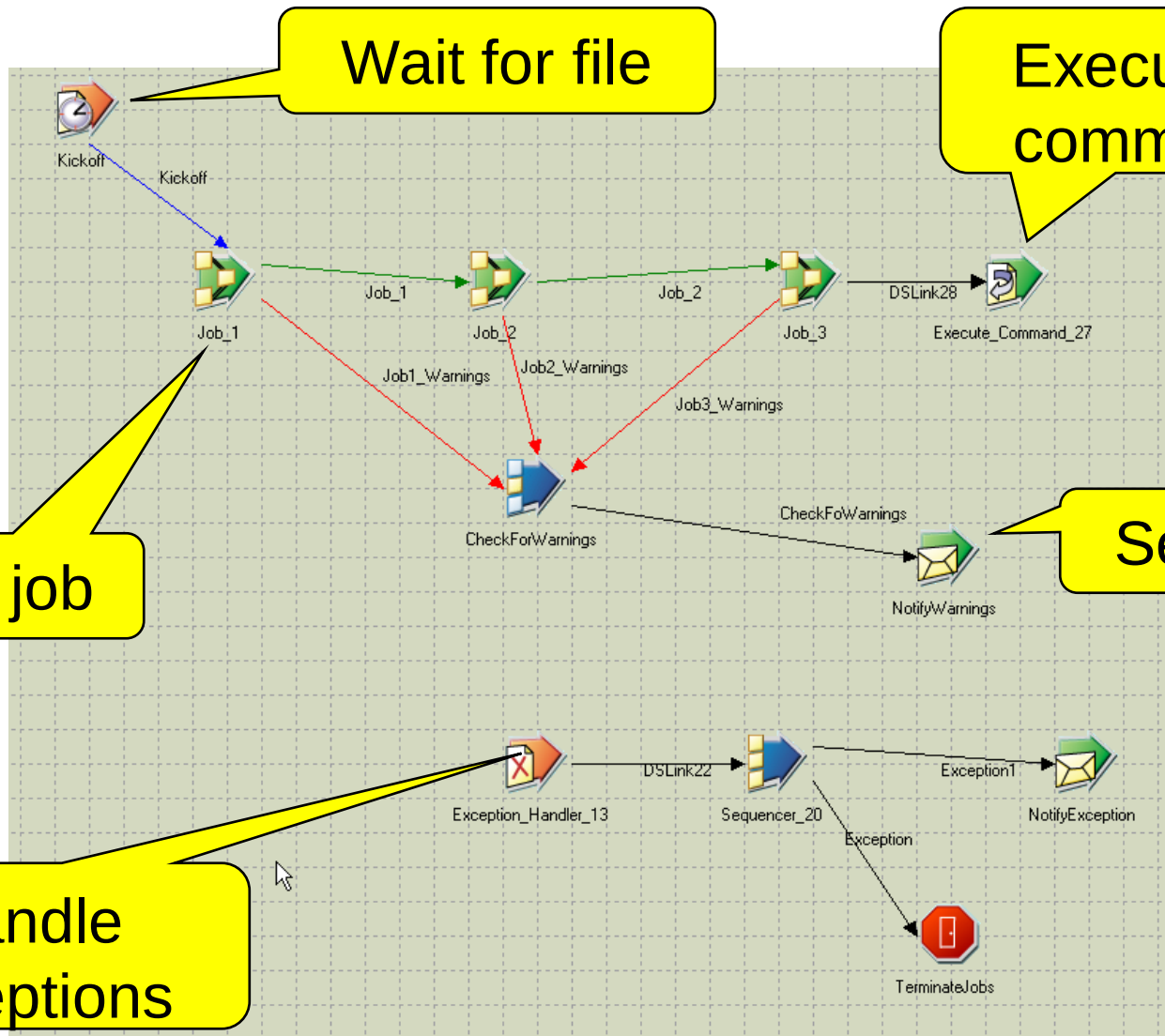
Wait for file

Execute a command

Send email

Run job

Handle exceptions



Sequence Properties - General

General | Parameters | Job control | Dependencies

Job version number:
50.0.0

☐ Allow Multiple Instance

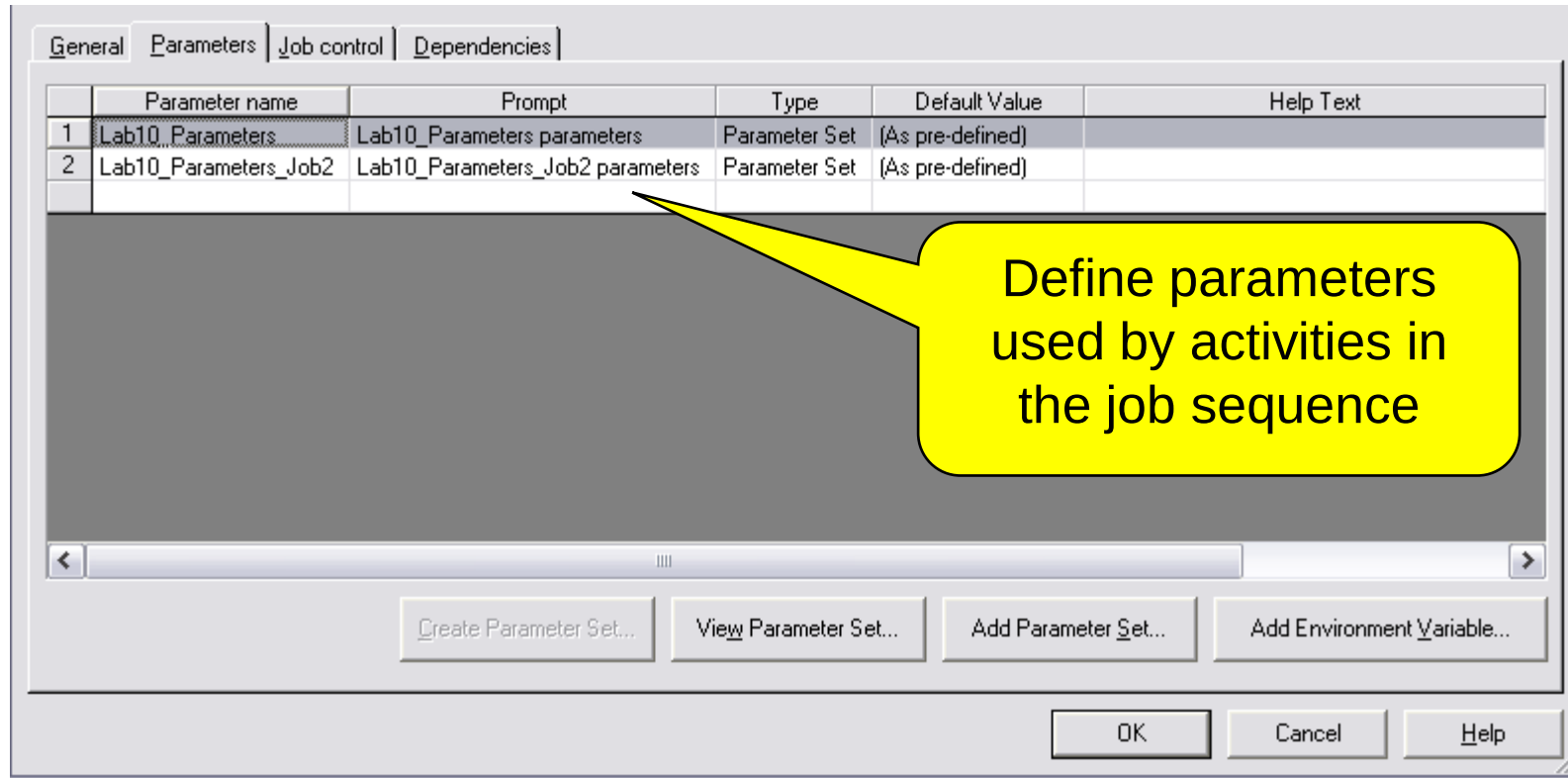
Restart

Compilation options

- ☒ Add checkpoints so sequence is restartable on failure
- ☒ Log warnings after activities that finish with status other than OK
- ☒ Automatically handle activities that fail
- ☒ Log report messages after each job run

Exception stage to handle aborts

Sequence Properties - Parameters



Job Activity Stage Properties

The screenshot shows the 'Warehouse_01 - Job Activity' dialog box. It has three tabs: 'General', 'Job', and 'Triggers'. The 'Job' tab is selected. The 'Job name' field contains 'Warehouse_01'. The 'Execution action' dropdown is set to 'Reset if required, then run'. The 'Parameters' table lists three parameters: '\$APT_CO...' with value '\$APT_CONFIG_FILE', '\$APT_DU...' with value '\$APT_DUMP_SCORE', and 'FileDate' with value 'FileDate'. There are buttons for 'Insert Parameter Value', 'Clear', and 'Clear All'. A 'Do not checkpoint run.' checkbox is also present. Three yellow callout boxes point to specific parts: 'Job to be executed' points to the job name, 'Execution mode' points to the execution action dropdown, and 'Job parameters to be passed' points to the parameters table.

Job to be executed

Execution mode

Job parameters to be passed

Name	Value Expression
\$APT_CO...	\$APT_CONFIG_FILE
\$APT_DU...	\$APT_DUMP_SCORE
FileDate	FileDate

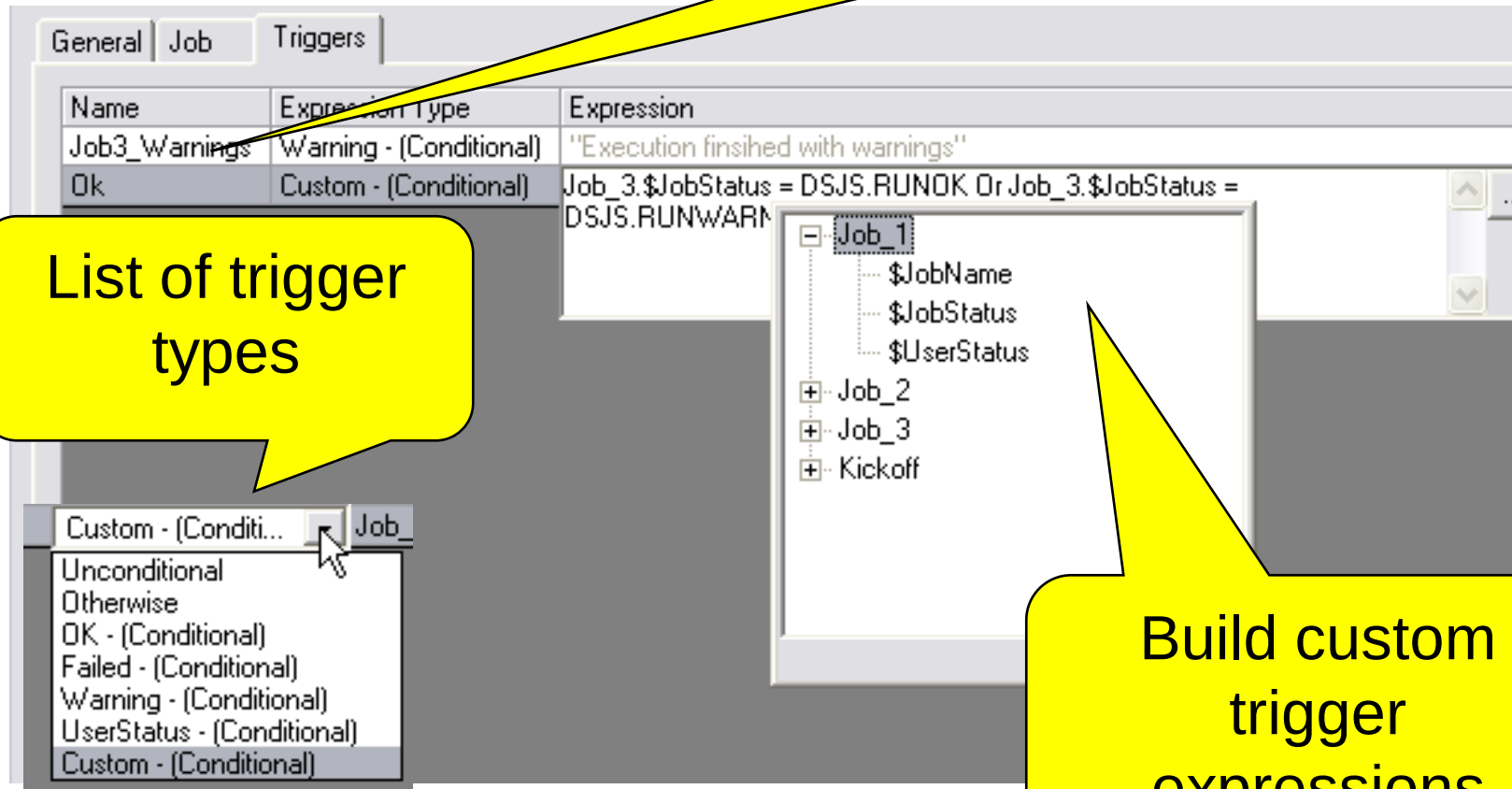
Type: Pathname
Prompt: Configuration file

Job Activity Trigger

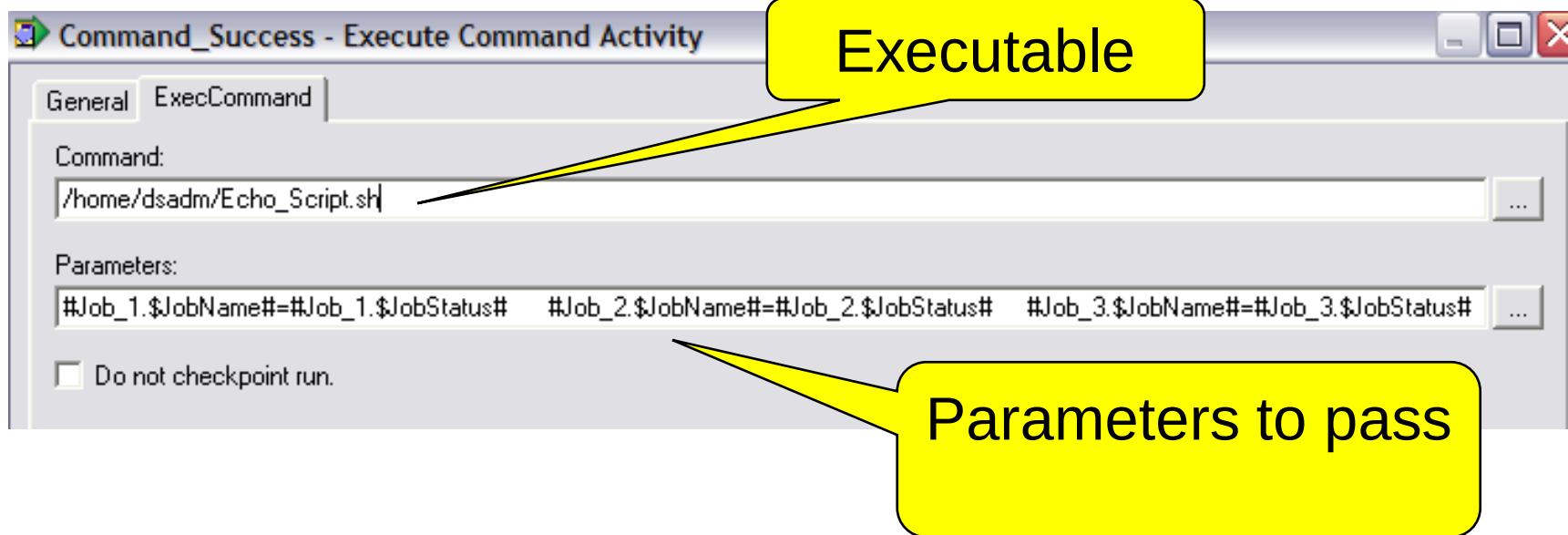
Output link names

List of trigger types

Build custom trigger expressions



Execute Command Stage



- Execute system commands, shell scripts, and other executables
- e.g. use to drop or rename database tables

Notification Activity Stage

NotifySuccess - Notification Activity

General | Notification

SMTP Mail server name:
NA-MSG-01

Senders email address:
lwilliams23@us.ibm.com

Recipients email address:
lwilliams23@us.ibm.com

Email subject:
Jobs executed successfully: #Job_1.\$JobName#, #Job_2.\$JobName#, #Job_3.\$JobName#

Attachments
/home/dsadm/ds324Files/Customers.txt

Email body:
All jobs ran successfully

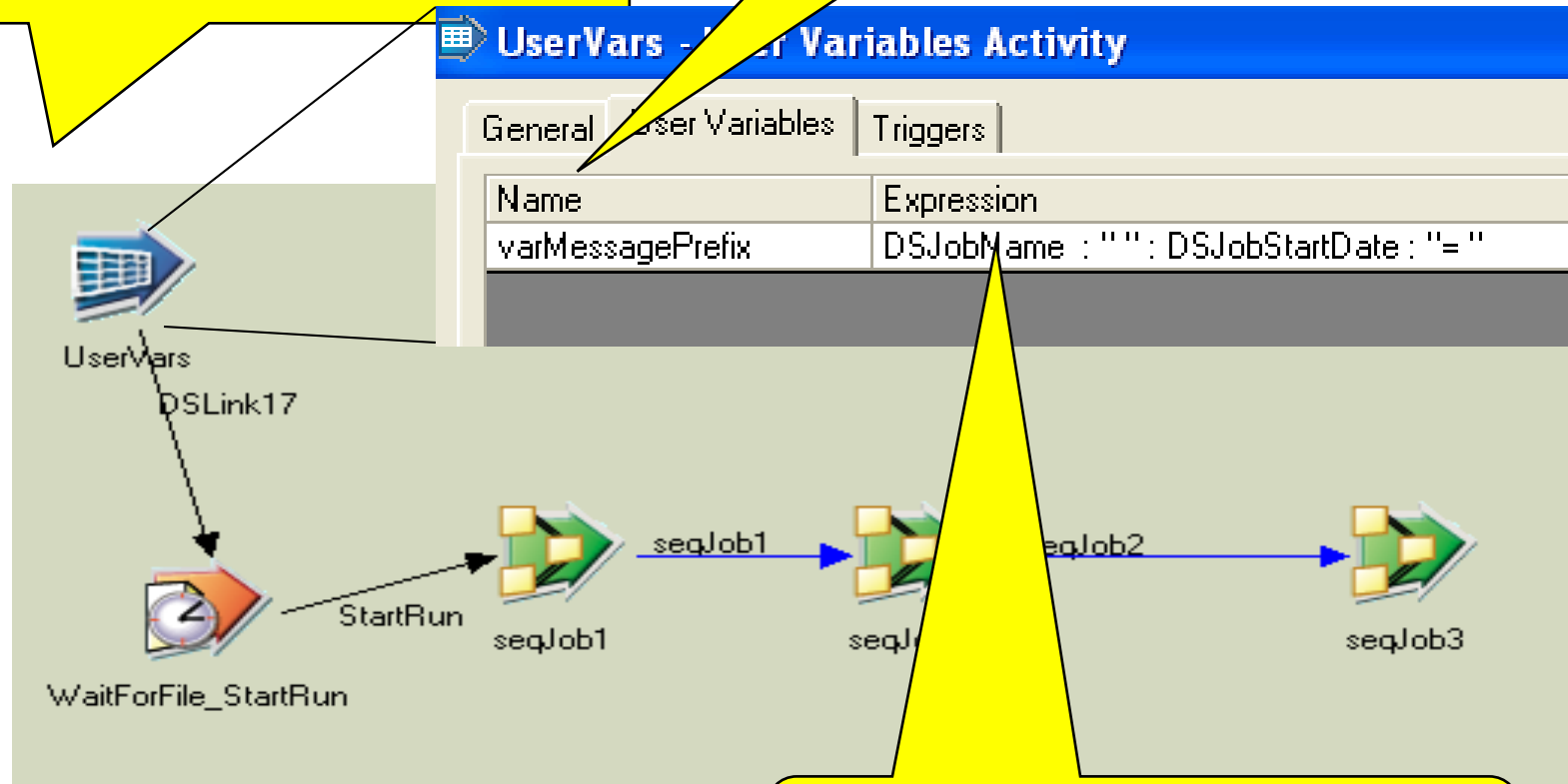
☒ Include job status in email
☐ Do not checkpoint run.

Include job
status info in
email body

User Variables Stage

User Variables stage

Define Variable



Expression defining the value for the variable

Referencing the User Variable

seqJob1 - Job Activity

General | Job | Triggers

Job name:
seqJob1

Execution action:
Reset if required, then run ☐ Do not checkpoint run.

Parameters

Name	Value Expression
NumRecs	RecCount1
\$APT_DUMP_SCORE	\$APT_DUMP_SCORE
PeekHeading	UserVars.varNamePrefix

Type: String
Prompt: NumRecs

Insert Parameter
Clear
Clear All
Set to Default
All to Default

Reference to the variable

Flow Control Stages

Flow Control Stages - Wait for File Stage

Kickoff - Wait for file Activity

General | **Wait For File** | Triggers

Filename:
/home/dsadm/StartRun

☐ Wait for file to appear
☒ Wait for file to disappear

Timeout length (hh:mm:ss):
00:00:00 ☒ Do not timeout

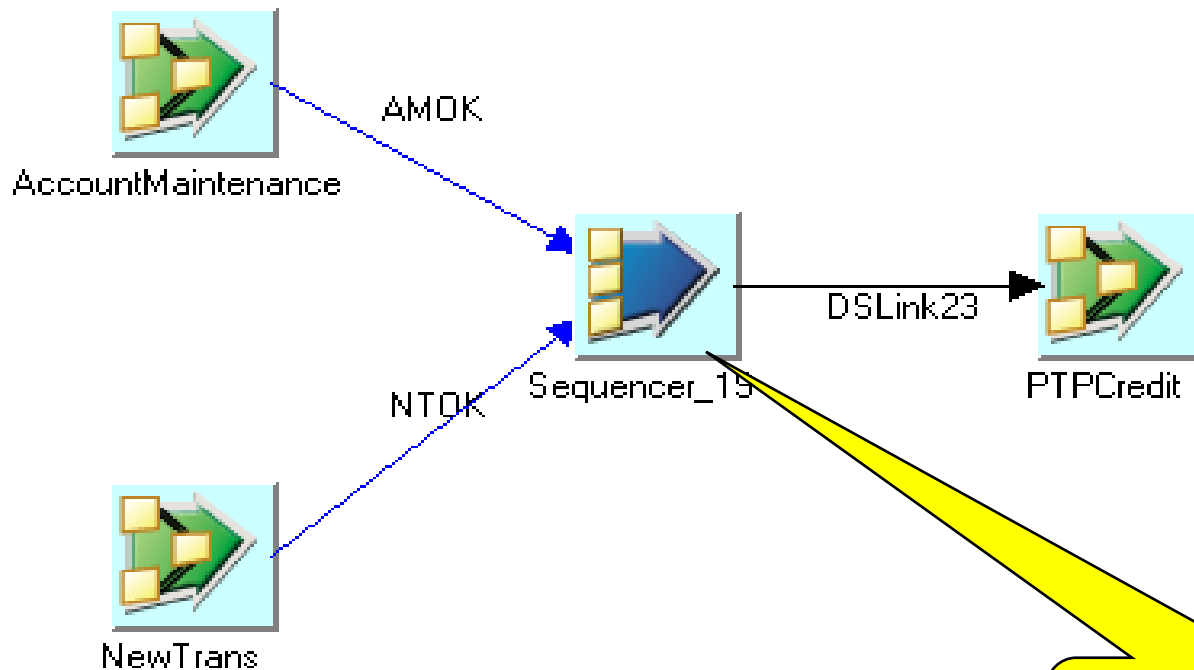
☐ Do not checkpoint run.

File

Options

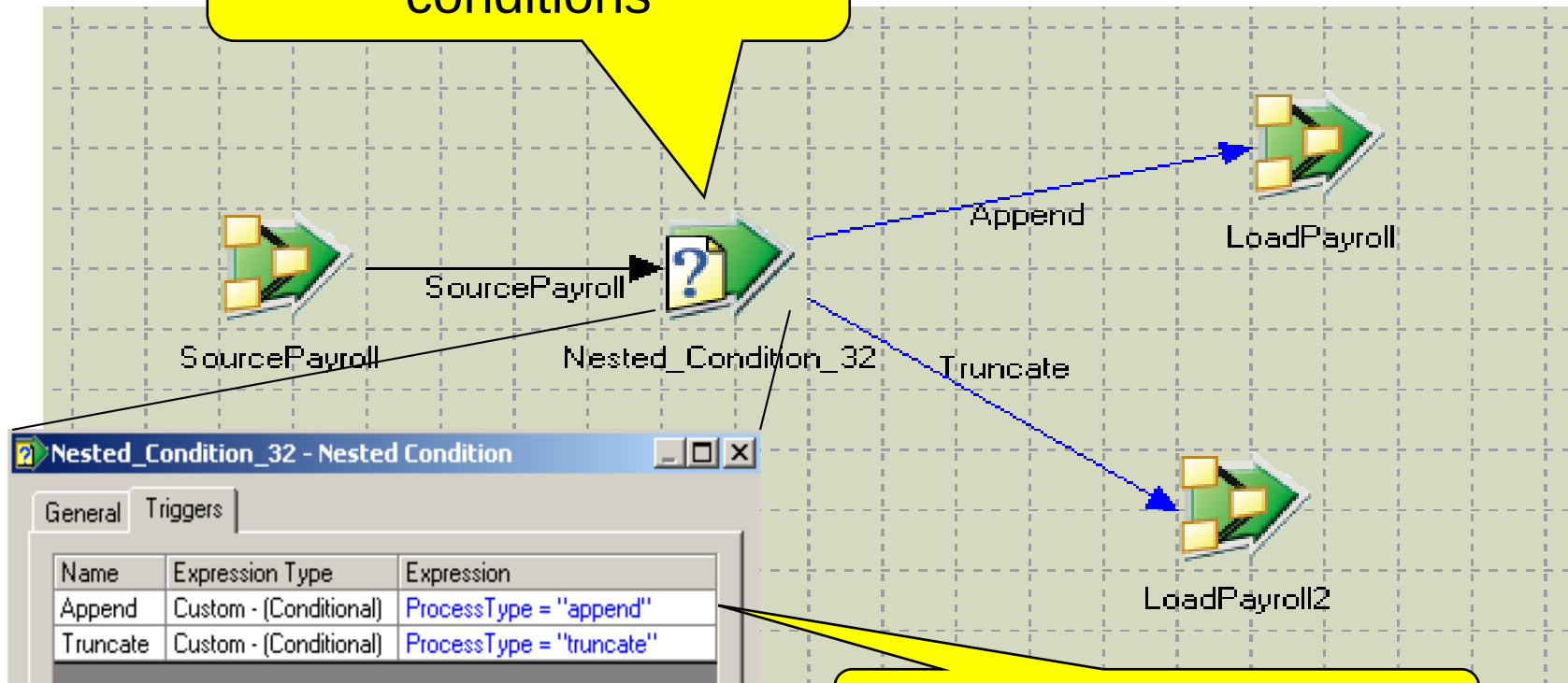
Flow Control Stages - Sequencer Stage

- Sequence multiple jobs using the Sequence stage



Flow Control Stages - Nested Condition Stage

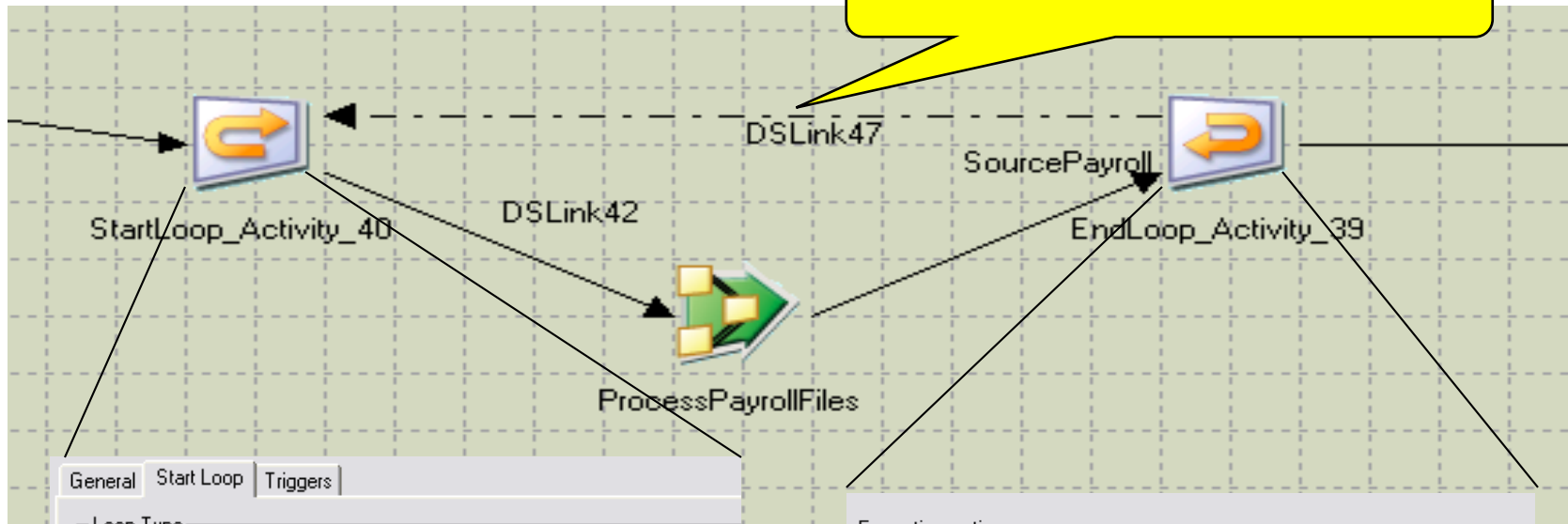
Fork based on trigger conditions



Trigger conditions

Flow Control Stages - Loop Stages

Reference link to start



General Start Loop Triggers

Loop Type

- ☐ Numeric Loop (For counter = n To nc Step inc)
- ☒ List Loop (For Each Thing in List)

Loop Definition

Delimited Values:

Payroll_File_01, Payroll_File_02, Payroll_File_03

Delimiter:

- ☒ Comma
- ☐ Space
- ☐ Other

Counter values

Execution action:

Run ☐ Do not checkpoint run.

Parameters

Name	Value Expression
FileName	StartLoop_Activity_40.\$Counter

Pass counter value

Error Handling

Handling Activities that Fail

General | Parameters | Job control | Dependencies

Category: ds434_New Job version num: 50.0.0

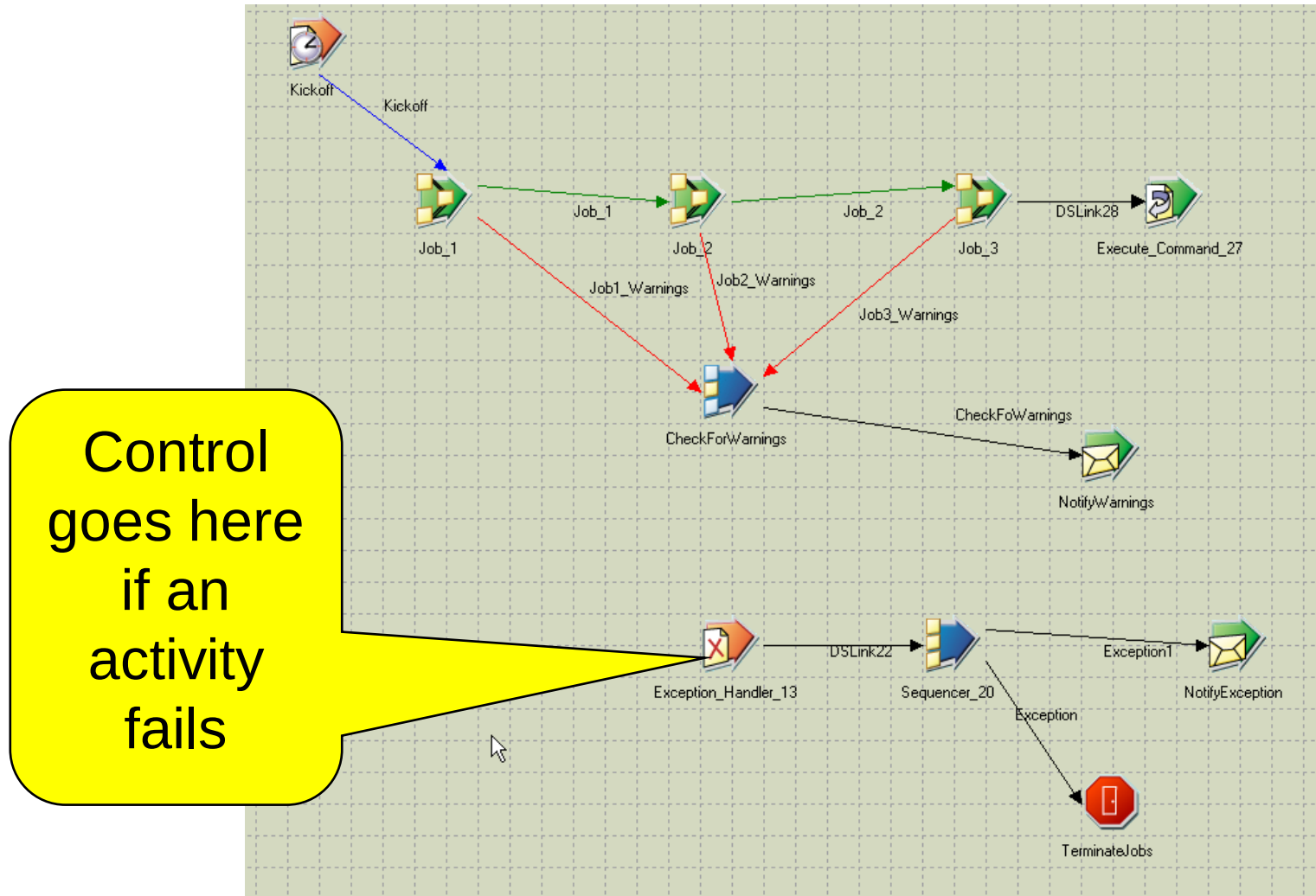
☐ Allow Multiple Instance

Compilation options

- ☒ Add checkpoints so sequence is restartable on failure
- ☒ Log warnings after activities that finish with status other than OK
- ☒ Automatically handle activities that fail
- ☒ Log report messages after each job run

Pass control to
Exception stage when
an activity fails

Exception Handler Stage



Restart

Enable Restart

General | Parameters | Job control | Dependencies

Category: ds434_New Job version number: 50.0.0

☐ Allow Multiple Instance

Compilation options

- ☒ Add checkpoints so sequence is restartable on failure
- ☒ Log warnings after activities that finish with status other than OK
- ☒ Automatically handle activities that fail
- ☒ Log report messages after each job run

Enable
checkpoints to
be added

Disable Checkpoint at a Stage

ProcessPayrollFiles - Job Activity

General Job Triggers

Job name:
SequencerJob_A

Execution action:
Run ☐ Do not checkpoint run.

Parameters

Name	Value Expression
FileName	StartLoop_Activity_40.\$Counter

Type: String
Prompt: FileName

Insert Parameter Value
Clear
Clear All
Set to Default
All to Default

Don't checkpoint this activity

Summing up – Why should I use a job sequence?

- Graphical interface
- You can propagate your jobs across environments like any other job (Design once, run anywhere)
- You can build programmatic controls, such as branching or looping
- You can specify different courses of action to take depending on whether a job in the sequence job succeeds or fails
- You can break up a long running job to many short jobs and using checkpoint restart to shorten reprocessing time when encounter error